Don't forget to put a link for our users to authenticate with Facebook in your sign-in page:

```
<div id="user_nav">
  <% if current_user %>
    Signed in as <strong><%= current_user.name %></strong>!
    <%= link_to "Sign out", signout_path, id: "sign_out" %>
  <% else %>
    <%= link_to "Sign in with Facebook", "/auth/facebook", id: "sign_
in" %>
  <% end %>
</div>
```

Actually, that was it for adding support for Facebook authentication. Adding support for Facebook is a lot easier, because we already have the logic in place when adding the Twitter authentication support. Of course, we can always make improvements to our application; the examples include support for scenarios where users can utilize multiprovider authentication in one account or support for LinkedIn and Google authentication.

Summary

By completing the guidance provided in this chapter, we will have successfully created an application that can authenticate its users using remote authentication with third-party providers. To explore this topic further, you can try to implement support for providers other than Twitter and Facebook, which was described in this chapter.

Another thing to explore is the database scheme and logic that was used in this chapter, which was created with an assumption that users can only have one remote authentication provider for their account. If they, say, want to sign in using a Facebook account or a Twitter account into the same account in our application, they won't be able to do so with the current configurations. To support that scheme, we may have to create a separate model for storing the `provider` and `uid` information. Our `User` model must also have a `has_many` relationship to this new model. Lastly, we also have to modify the logic in our application accordingly.

www.it-ebooks.info